



TUK Logo

TECHNICAL UNIVERSITY OF KENYA

FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS

**LINDAJAMII: A FULL-STACK COMMUNITY SAFETY AND INFORMATION
PLATFORM**

PRESENTED BY:

STUDENT NAME: [Your Name]

REG NO: [Your Registration Number]

SUPERVISED BY:

[Supervisor's Name]

**PROPOSAL SUBMITTED TO THE SCHOOL OF COMPUTING AND INFORMATION
TECHNOLOGY IN PARTIAL FULFILLMENT FOR THE **BACHELOR OF / DIPLOMA IN****

**TECHNOLOGY IN COMPUTER TECHNOLOGY PROJECT OF THE TECHNICAL
UNIVERSITY OF KENYA**

SUBMISSION DATE:

14 May 2026

DECLARATION

I, the undersigned, declare that this project proposal is my original work and has not been submitted to any other university or institution for the award of a degree or diploma. All sources of information have been specifically acknowledged by means of references.

Student Name: _____

Signature: _____

Date: _____

This project proposal has been submitted for examination with my approval as the University Supervisor.

Supervisor Name: _____

Signature: _____

Date: _____

DEDICATION

This project is dedicated to the communities striving for safety, transparency, and resilience. It is also dedicated to my family and friends for their unwavering support and encouragement throughout my academic journey.

ACKNOWLEDGEMENT

I would like to express my profound gratitude to my supervisor, [Supervisor's Name], for their invaluable guidance, constructive criticism, and continuous support during the formulation of this proposal. I also extend my thanks to the faculty members of the School of Computing and Information Technology at the Technical University of Kenya for equipping me with the necessary knowledge and skills. Finally, I am grateful to my peers for their collaborative spirit and insightful discussions.

LIST OF ABBREVIATIONS

- **API:** Application Programming Interface
- **CRUD:** Create, Read, Update, Delete
- **DFD:** Data Flow Diagram
- **ERD:** Entity-Relationship Diagram
- **HTML:** HyperText Markup Language
- **HTTP:** Hypertext Transfer Protocol
- **JSON:** JavaScript Object Notation
- **JWT:** JSON Web Token
- **REST:** Representational State Transfer
- **SCIT:** School of Computing and Information Technology
- **SPA:** Single Page Application
- **SQL:** Structured Query Language
- **STK:** Sim Tool Kit
- **UI:** User Interface
- **UML:** Unified Modeling Language

LIST OF FIGURES

- Figure 1: Proposed System Architecture

LIST OF TABLES

- Table 1: Project Schedule (Gantt Chart Summary)
- Table 2: Estimated Budget and Resources
- Table 3: Comparison of Existing Systems
- Table 4: Hardware Specifications
- Table 5: Software Specifications

ABSTRACT

The rapid urbanization and increasing complexity of modern societies have highlighted significant gaps in community safety and information dissemination. Traditional communication channels are often fragmented, leading to delayed incident reporting and a lack of verified, real-time information regarding local threats, weather changes, and relevant news. This project proposes the development of “LindaJamii,” a comprehensive, full-stack web platform designed to address these challenges. LindaJamii will provide a centralized hub for real-time incident reporting, community alerts, and an integrated InfoHub delivering localized weather forecasts and curated news (medical, geopolitical, climatic). Furthermore, the platform will integrate the Safaricom Daraja API to facilitate secure M-Pesa donations, empowering communities to financially support local safety initiatives. The system will be built using a robust, multi-language microservices architecture, utilizing C for high-performance utilities, Python (Flask) for incident and information APIs, and Java (Spring Boot) for secure neighbourhood and patrol management, all connected to a modern HTML/CSS/JavaScript frontend. This proposal outlines the background, objectives, methodology, and system analysis required to successfully implement LindaJamii, aiming to foster a more informed, responsive, and resilient community environment.

TABLE OF CONTENTS

1. [CHAPTER ONE: INTRODUCTION](#)

- 1.1 [Introduction](#)
- 1.2 [Background of the Study](#)
- 1.3 [Problem Statement](#)
- 1.4 [Objectives](#)
- 1.5 [Justification](#)
- 1.6 [Scope of the Study](#)
- 1.7 [Limitations of the Proposed System](#)
- 1.8 [Project Risk and Mitigation](#)
- 1.9 [Project Schedule](#)
- 1.10 [Budget and Resources](#)

2. [CHAPTER TWO: LITERATURE REVIEW](#)

- 2.1 [Reviewed Similar Systems](#)
- 2.2 [Tools and Methodologies used in Reviewed Systems](#)
- 2.3 [Gaps in the Existing Systems and the Proposed Solution](#)
- 2.4 [The Proposed Solution](#)

3. [CHAPTER THREE: METHODOLOGY](#)

- 3.1 [Methodology and Tools](#)
- 3.2 [Source of Data](#)
- 3.3 [Data Collection Methods](#)
- 3.4 [Resources Required / Materials](#)

4. [CHAPTER FOUR: SYSTEM ANALYSIS AND REQUIREMENT MODELING](#)

- 4.1 [Introduction to System Analysis](#)
- 4.2 [Objectives of System Analysis](#)
- 4.3 [Problem Definition](#)
- 4.4 [Feasibility Study](#)

- 4.5 [System Analysis Tools](#)
- 4.6 [System Investigation](#)
- 4.7 [System Analysis](#)

5. [BIBLIOGRAPHY](#)

CHAPTER ONE: INTRODUCTION

1.1 Introduction

The concept of community safety has evolved significantly with the advent of digital technology. However, many local communities still struggle with fragmented communication, delayed responses to incidents, and a lack of access to verified, real-time information. LindaJamii, a Swahili term translating to “Protect Community,” is conceptualized as a comprehensive digital platform designed to bridge these gaps. It aims to empower residents by providing a centralized hub for incident reporting, safety alerts, and critical information dissemination.

This project will involve the design and implementation of a full-stack web application. The architecture will employ a modern, distributed approach, utilizing a multi-language backend to handle specific tasks efficiently: a C-based microservice for low-level utilities, a Python (Flask) API for rapid data processing and external integrations, and a Java (Spring Boot) API for robust, secure enterprise logic. The frontend will be a responsive Single Page Application (SPA) providing an intuitive user interface. Key features will include real-time incident tracking, an InfoHub for weather and news, and an integrated M-Pesa payment gateway to facilitate community donations.

1.2 Background of the Study

1.2.1 Background of the Organization

LindaJamii is proposed as a versatile platform intended for adoption by neighborhood associations, local security groups (such as Nyumba Kumi initiatives in Kenya), and community-based organizations. Historically, these groups have relied on physical meetings, notice boards, or basic messaging apps (like WhatsApp or Telegram) to coordinate activities. While these methods have fostered community spirit, they lack the structure required for efficient incident management, data analysis, and secure financial contributions. The growth of these organizations necessitates a dedicated

system that can handle complex workflows, ensure data privacy, and provide actionable intelligence to community leaders and residents alike.

1.2.2 Overview of Existing System

Currently, community safety and information sharing are largely decentralized. Residents might use a WhatsApp group to report a suspicious person, check a separate weather app for forecasts, and rely on mainstream media for news.

Procedures in the Existing System:

1. An incident occurs.
2. A resident types a message in a community chat group.
3. The message may be missed if the chat is highly active.
4. Community leaders manually record the incident if they see it.
5. Financial contributions for community projects are collected via personal mobile money transfers, lacking transparency and automated receipting.

Disadvantages of the Existing System:

- **Information Overload and Loss:** Critical alerts are easily buried in informal chat groups.
- **Lack of Verification:** It is difficult to verify the authenticity of reports shared informally.
- **Inefficient Resource Allocation:** Without centralized data, it is challenging to identify crime hotspots or trends.
- **Financial Opacity:** Manual collection of funds lacks transparency and accountability.
- **Fragmented Information:** Residents must use multiple apps to stay informed about their environment (weather, news, local alerts).

1.2.3 Overview of the Proposed System

The proposed LindaJamii system will centralize these functions into a single, cohesive web application.

Procedures in the Proposed System:

1. A user logs into the LindaJamii dashboard.
2. To report an incident, the user fills out a structured form (category, severity, location).
3. The system processes the report, updates the central database, and immediately displays it on the community feed.
4. The InfoHub automatically fetches and displays real-time weather and relevant news.
5. Users can securely donate to community funds via an integrated M-Pesa STK push directly from the platform.

Benefits of the Proposed System:

- **Centralized Communication:** A single source of truth for community safety and information.
- **Structured Data:** Incident reports are categorized and easily analyzable.
- **Real-time Awareness:** The InfoHub keeps residents informed of environmental and geopolitical factors affecting their safety.
- **Transparent Funding:** Integrated M-Pesa gateway ensures secure, trackable donations.
- **Scalability:** The microservices architecture allows the system to grow and handle increased traffic efficiently.

1.3 Problem Statement

Despite the proliferation of communication technologies, local communities lack a dedicated, integrated platform for managing safety and disseminating critical information. The reliance on fragmented, informal channels (such as social media groups) results in delayed incident reporting, unverified information, and a lack of actionable data for community leaders. Furthermore, the absence of a transparent, integrated mechanism for community funding hinders the sustainability of local safety initiatives. This project addresses the problem of inefficient community coordination by developing LindaJamii, a unified system that streamlines incident reporting, provides real-time environmental and news updates, and facilitates secure financial contributions, thereby enhancing overall community resilience.

1.4 Objectives

1.4.1 Project Goal (Major Objective)

The primary goal of this project is to design, develop, and implement LindaJamii, a full-stack, multi-language web platform that enhances community safety and awareness through centralized incident reporting, real-time information dissemination, and integrated mobile payment capabilities.

1.4.2 General Objectives

- To develop a secure and scalable distributed backend architecture.
- To create an intuitive and responsive user interface for community members.
- To integrate external APIs to provide real-time, relevant data to users.
- To implement a secure financial transaction module for community support.

1.4.3 Specific Objectives

- To design and implement a relational database schema to manage users, incidents, patrols, and notifications.
- To develop a Java (Spring Boot) REST API with Spring Security for managing user authentication and neighborhood patrol schedules.
- To develop a Python (Flask) REST API to handle incident reporting logic and integrate external Weather and News APIs.
- To develop a C-based microservice to handle high-performance utility tasks such as data hashing and system health monitoring.
- To integrate the Safaricom Daraja API (STK Push) within the Python backend to process M-Pesa donations.
- To build a modern HTML/CSS/JavaScript frontend featuring a 5-step onboarding flow, an interactive dashboard, and dedicated components for the InfoHub and Donation gateway.

1.5 Justification

The realization of the LindaJamii project will bring significant value to local communities. By transitioning from informal communication methods to a structured platform, communities will benefit from faster response times to incidents and a clearer understanding of local safety trends. The integration of the InfoHub ensures that residents are not only aware of immediate local threats but also broader environmental and health factors that could impact them. The M-Pesa integration is particularly crucial in the Kenyan context, providing a familiar, secure, and transparent method for funding community initiatives, such as paying neighborhood watch personnel or installing streetlights. Technically, the project presents a challenging and highly relevant exercise in building distributed systems, utilizing a polyglot architecture that reflects modern enterprise software development practices.

1.6 Scope of the Study

The project will cover the end-to-end development of the LindaJamii web platform. This includes:

- **Frontend:** Development of the user interface using HTML, CSS, and JavaScript, focusing on responsive design and user experience.
- **Backend:** Implementation of three distinct services (Java, Python, C) and their inter-communication.
- **Database:** Design and implementation of a PostgreSQL/MySQL relational database.
- **Integrations:** Connection to the Safaricom Daraja API for payments, OpenWeatherMap for weather data, and NewsAPI for curated news. The scope is limited to the web application; native mobile applications (Android/iOS) will not be developed in this phase. The system will be designed to serve a localized community structure (e.g., a specific neighborhood or estate) as a proof of concept.

1.7 Limitations of the Proposed System

- **Internet Dependency:** The system requires a stable internet connection to function, which may be a constraint in areas with poor connectivity.
- **Third-Party API Reliance:** The functionality of the InfoHub and Donation gateway depends entirely on the uptime and rate limits of external providers (Safaricom, OpenWeatherMap, NewsAPI).
- **Hardware Constraints:** The initial deployment will be on a single server environment, which may limit the number of concurrent users it can handle before requiring a distributed cloud deployment.

1.8 Project Risk and Mitigation

Risk	Description	Mitigation Strategy
API Integration Failure	Changes in third-party API endpoints or authentication methods (e.g., Daraja API).	Implement robust error handling and fallback mechanisms. Regularly monitor API documentation for updates.
Security Vulnerabilities	Unauthorized access to user data or fraudulent donation requests.	Utilize Spring Security for robust authentication (JWT), bcrypt for password hashing, and validate all inputs.
Scope Creep	The project expands beyond the initial objectives, causing delays.	Strictly adhere to the defined scope and specific objectives. Any new features will be documented for future versions.
Time Constraints	Inability to complete the multi-language backend within the academic timeframe.	Utilize agile development methodologies, prioritizing core features (MVP) first before adding complex integrations.

1.9 Project Schedule

The project will be executed over a 12-week period, following an iterative development approach.

Phase	Activity	Duration
1	Requirement Gathering & System Analysis	2 Weeks
2	System Design (Architecture, Database, UI/UX)	2 Weeks
3	Backend Development (Java, Python, C)	3 Weeks
4	Frontend Development & API Integration	2 Weeks
5	External API Integration (M-Pesa, Weather, News)	1 Week
6	System Testing & Debugging	1 Week
7	Documentation & Final Presentation Preparation	1 Week

(A detailed Gantt chart will be provided in the Appendix of the final report).

1.10 Budget and Resources

This project is primarily software-based and will utilize open-source technologies, minimizing direct financial costs.

Item	Description	Estimated Cost (KES)
Hardware	Personal Computer (Core i5, 8GB RAM, 256GB SSD)	Existing
Software	IDEs (VS Code, IntelliJ), PostgreSQL, Git	Free / Open Source
Hosting	Cloud VPS (e.g., DigitalOcean, AWS Free Tier) for deployment	2,000 / month
Internet	Data bundles for research and API communication	3,000
Miscellaneous	Printing, binding of final reports	1,500
Total		6,500

CHAPTER TWO: LITERATURE REVIEW

2.1 Reviewed Similar Systems

Several systems exist that attempt to address community safety and communication, though often in a fragmented manner.

1. **Nextdoor:** A widely used hyper-local social networking service for neighborhoods. It allows users to share information, organize events, and report safety concerns.
2. **Citizen:** A mobile app that sends location-based safety alerts in real-time. It relies on monitoring emergency scanner radios and user-generated reports.
3. **WhatsApp/Telegram Groups:** The most common informal method used by communities in Kenya (e.g., Nyumba Kumi groups) for rapid communication and alerts.

2.2 Tools and Methodologies used in Reviewed Systems

- **Nextdoor:** Utilizes a monolithic architecture transitioning to microservices, heavily relying on geospatial data processing to verify user addresses and group them into neighborhoods. *Advantage:* High user engagement. *Disadvantage:* Prone to gossip and unverified information; lacks integrated financial tools for community projects.
- **Citizen:** Employs real-time data streaming technologies (like WebSockets or Kafka) to push alerts instantly to users. *Advantage:* Extremely fast alert dissemination. *Disadvantage:* Can cause unnecessary panic; lacks broader community building features (like news or weather).
- **WhatsApp Groups:** Uses standard instant messaging protocols. *Advantage:* Ubiquitous and easy to use. *Disadvantage:* Unstructured data, impossible to analyze trends, no integration with external APIs (like M-Pesa or Weather).

2.3 Gaps in the Existing Systems and the Proposed Solution

The primary gap identified in the reviewed systems is the lack of a **holistic, integrated approach** tailored to the specific needs of developing communities, particularly in regions like East Africa.

- **Financial Integration:** None of the reviewed safety platforms natively integrate local mobile money solutions (like M-Pesa) to fund community policing or infrastructure.
- **Contextual Information:** While apps like Citizen provide crime alerts, they do not provide contextual environmental data (weather, health news) which are equally critical to community resilience.
- **Structured Reporting vs. Accessibility:** WhatsApp is accessible but unstructured. Nextdoor is structured but often too broad.

2.4 The Proposed Solution

LindaJamii bridges these gaps by offering a tailored, full-stack solution. It provides the structured incident reporting of a dedicated app, combined with the contextual awareness of an InfoHub (Weather/News), and crucially, integrates Safaricom's Daraja API for seamless M-Pesa donations. By utilizing a microservices architecture (Java, Python, C), the system ensures that high-load tasks (like real-time alerts) do not interfere with secure transactions or data processing, providing a robust and scalable platform superior to informal chat groups.

CHAPTER THREE: METHODOLOGY

3.1 Methodology and Tools

This project will adopt the **Agile Software Development Methodology**, specifically utilizing the Scrum framework. Agile is chosen because it promotes iterative development, allowing for continuous feedback and flexibility in accommodating changes, which is crucial when integrating multiple third-party APIs (M-Pesa, Weather, News) that may require adjustments during development.

Phases of the Agile Methodology:

1. **Planning & Requirements:** Defining the product backlog based on the objectives.
2. **Design:** Architecting the multi-language backend and designing the UI/UX.
3. **Development (Sprints):** Developing features in 2-week sprints (e.g., Sprint 1: Java Backend & Auth; Sprint 2: Python API & InfoHub; Sprint 3: Frontend Integration).
4. **Testing:** Continuous integration and testing during each sprint.
5. **Deployment:** Deploying the application to a cloud environment.

3.2 Source of Data

- **Primary Data:** Data generated by users interacting with the system (incident reports, user profiles, donation records).
- **Secondary Data:** Data fetched from external APIs:
 - Weather data from OpenWeatherMap API.
 - News articles from NewsAPI.
 - Transaction status data from Safaricom Daraja API.

3.3 Data Collection Methods

For the purpose of system requirement gathering and validation:

- **Document Inspection:** Reviewing existing literature on community policing and digital safety platforms.
- **Observation:** Observing how current community groups (e.g., WhatsApp groups) handle information and financial contributions to understand user behavior and pain points.
- **Prototyping:** Developing a rapid prototype of the UI to gather feedback on usability before full implementation.

3.4 Resources Required / Materials

3.4.1 Hardware Specifications

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5
Memory (RAM)	4 GB	8 GB or higher
Storage	100 GB HDD	256 GB SSD
Internet	5 Mbps connection	20 Mbps connection

3.4.2 Software Specifications

Component	Specification
Operating System	Ubuntu Linux 22.04 (Development & Server)
Programming Languages	Java 17, Python 3.11, C11, JavaScript (ES6+)
Frameworks	Spring Boot 3.x (Java), Flask (Python)
Database	PostgreSQL 14+
Web Server	Nginx / Python HTTP Server (Frontend hosting)
Tools	Git, Maven, pip, gcc, Postman (API testing)

CHAPTER FOUR: SYSTEM ANALYSIS AND REQUIREMENT MODELING

4.1 Introduction to System Analysis

System analysis involves a detailed examination of the current methods used for community communication and the formulation of a new, integrated digital system. This chapter outlines the feasibility of LindaJamii and models the requirements using standard analysis tools to ensure the proposed system meets the defined objectives.

4.2 Objectives of System Analysis

- To determine the technical, economic, and operational feasibility of LindaJamii.
- To clearly define the functional and non-functional requirements of the system.
- To model the flow of data and system processes using Context Diagrams and Data Flow Diagrams (DFDs).

4.3 Problem Definition

The core problem is the inefficiency and fragmentation of current community safety communication. Information is siloed, incident reporting is unstructured, and community funding mechanisms are opaque. The proposed system must solve this by providing a unified interface that handles structured data entry (incidents), external data retrieval (news/weather), and secure financial transactions (M-Pesa).

4.4 Feasibility Study

- **Technical Feasibility:** The project is technically feasible. The chosen technologies (Java, Python, C, HTML/JS) are well-documented, open-source, and

widely supported. The integration of RESTful APIs is a standard industry practice.

- **Economic Feasibility:** The project is economically viable. Development relies on open-source software. The only costs incurred are for cloud hosting and internet access, which are minimal and within the student budget.
- **Operational Feasibility:** The system is designed to be user-friendly, requiring minimal training. If adopted by a community, it will streamline operations, making it highly operationally feasible.
- **Schedule Feasibility:** The 12-week timeframe is sufficient to develop the core MVP (Minimum Viable Product) using Agile methodologies, provided scope creep is managed.

4.5 System Analysis Tools

The following tools will be used to model the system:

- **Context Diagrams:** To show the system boundaries and interactions with external entities (Users, M-Pesa API, Weather API).
- **Data Flow Diagrams (DFDs):** To illustrate how data moves through the system processes.
- **Entity-Relationship Diagrams (ERDs):** To model the database structure.

4.6 System Investigation

4.6.1 Introduction

Investigation involves understanding the exact data points required to make the system functional.

4.6.2 Data Collection

Data regarding required features was collected by analyzing the shortcomings of existing platforms (like WhatsApp groups) where messages lack structure (e.g., missing location data for an incident).

4.6.3 Fact Recording

Program Requirements:

- **Input Requirements:** User registration details, incident details (type, location, description), donation amount, phone number.
- **Output Requirements:** Dashboard feed of incidents, weather forecast display, news list, payment success/failure notifications.
- **Process Requirements:** User authentication (JWT), routing requests to appropriate microservices, initiating STK push, fetching external API data.

4.7 System Analysis

The analysis of the requirements dictates a distributed architecture.

Data Flow Representation:

1. **User Entity** interacts with the **Frontend UI**.
2. Frontend sends authentication requests to the **Java API**.
3. Frontend sends incident reports and external data requests to the **Python API**.
4. The Python API communicates with the **M-Pesa API** for donations and **OpenWeather/NewsAPI** for the InfoHub.
5. The Python API may call the **C Microservice** for specific high-speed data hashing or validation tasks.

(Note: Detailed Context Diagrams and DFDs will be included in the final project documentation).

BIBLIOGRAPHY

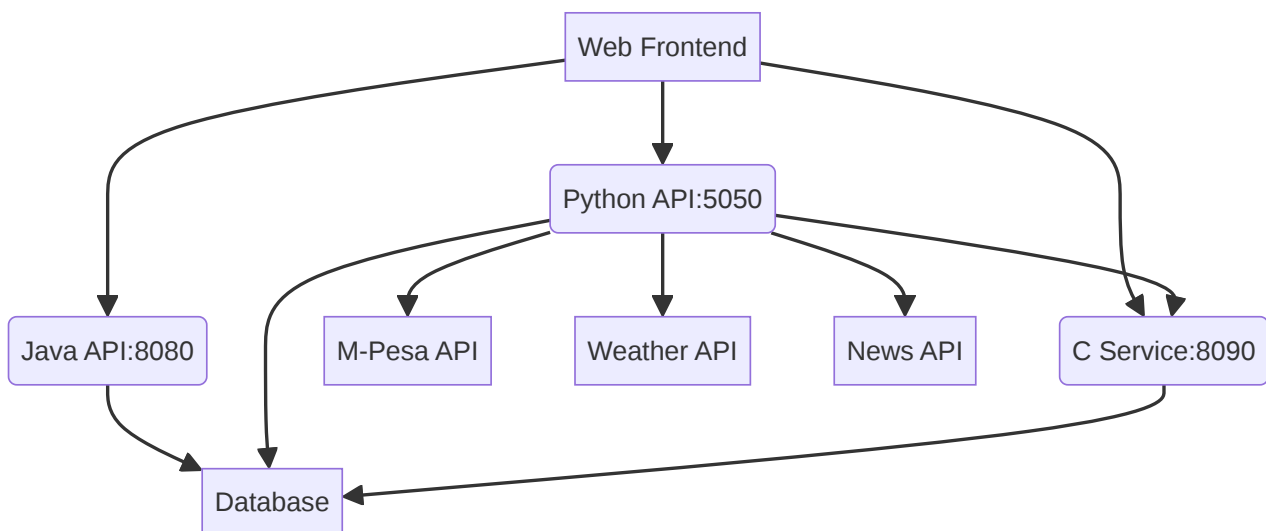
1. Sommerville, I. (2015). *Software Engineering*. 10th ed. Pearson.
2. Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
3. Safaricom Developer Portal. (2023). *Daraja API Documentation*. [Online] Available at: <https://developer.safaricom.co.ke/> [Accessed 14 May 2026].
4. OpenWeatherMap. (2023). *Weather API Documentation*. [Online] Available at: <https://openweathermap.org/api> [Accessed 14 May 2026].
5. NewsAPI. (2023). *News API Documentation*. [Online] Available at: <https://newsapi.org/docs> [Accessed 14 May 2026].
6. Spring Framework Documentation. (2023). *Spring Boot Reference Guide*. [Online] Available at: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> [Accessed 14 May 2026].
7. Pallets Projects. (2023). *Flask Documentation*. [Online] Available at: <https://flask.palletsprojects.com/> [Accessed 14 May 2026].

CHAPTER FIVE: SYSTEM DESIGN AND IMPLEMENTATION

5.1 System Architecture

LindaJamii employs a **microservices architecture** to ensure scalability, maintainability, and flexibility. This polyglot approach leverages the strengths of different programming languages for specific tasks. The system is composed of a modern web frontend and three distinct backend services (C, Python, Java) that communicate via RESTful APIs.

Figure 1: Proposed System Architecture



- **Web Frontend:** A Single Page Application (SPA) built with HTML, CSS, and JavaScript, providing the user interface and interacting with the backend APIs.
- **Python API (Flask):** Handles incident reporting, community alerts, and integrates with external APIs (M-Pesa, Weather, News). It also serves as an orchestrator for certain tasks, potentially calling the C service.
- **Java API (Spring Boot):** Manages user authentication, authorization (via Spring Security), neighbourhood registries, and patrol scheduling. It provides robust,

enterprise-grade services.

- **C Micro-Service:** A high-performance, low-level service for specific utility functions like hashing, health checks, and potentially real-time data processing where speed is critical.
- **Database:** A PostgreSQL/MySQL relational database serves as the persistent storage for all application data.

5.2 Database Design

The database schema is designed to support the core functionalities of a community safety and information platform, drawing inspiration from social media-like interactions for community engagement. The Entity-Relationship Diagram (ERD) below illustrates the relationships between key entities.

****Table Definitions:****

```sql

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  username VARCHAR(255) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  phone_number VARCHAR(20),  
  profile_picture_url TEXT,  
  bio TEXT,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  is_verified BOOLEAN DEFAULT FALSE,  
  role VARCHAR(50) DEFAULT 'user'  
);
```

```
CREATE TABLE posts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  content TEXT NOT NULL,  
  media_url TEXT,  
  post_type VARCHAR(50) DEFAULT 'text',  
  location VARCHAR(255),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE comments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  content TEXT NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE followers (  
  follower_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  followed_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (follower_id, followed_id)  
);
```

```
CREATE TABLE likes (  

```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
UNIQUE (post_id, user_id) -- A user can only like a post once
);
```

```
CREATE TABLE chats (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chat_name VARCHAR(255),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chat_id UUID NOT NULL REFERENCES chats(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type VARCHAR(50) NOT NULL,
  content TEXT NOT NULL,
  is_read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE incidents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reporter_id UUID NOT NULL REFERENCES users(id) ON DELETE SET NULL,
  incident_type VARCHAR(100) NOT NULL,
  description TEXT,
  location VARCHAR(255),
  status VARCHAR(50) DEFAULT 'reported',
  severity VARCHAR(50) DEFAULT 'medium',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE alerts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  alert_type VARCHAR(100) NOT NULL,
```

```

    message TEXT NOT NULL,
    geographical_scope VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE neighbourhoods (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) UNIQUE NOT NULL,
    description TEXT,
    location_coords VARCHAR(255), -- e.g., "-1.286389, 36.817223"
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE patrols (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    neighbourhood_id UUID NOT NULL REFERENCES neighbourhoods(id) ON DELETE
CASCADE,
    patrol_leader_id UUID REFERENCES users(id) ON DELETE SET NULL,
    start_time TIMESTAMP WITH TIME ZONE NOT NULL,
    end_time TIMESTAMP WITH TIME ZONE NOT NULL,
    status VARCHAR(50) DEFAULT 'scheduled',
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

5.3 Frontend Design

The frontend is designed as a modern Single Page Application (SPA) with a clear, intuitive user flow and responsive design principles. The user experience is structured around a 5-step application flow:

1. **Splash Screen:** Initial loading screen, introducing the app.
2. **Sign Up:** User registration with email/phone and password.
3. **Verify:** Account verification (e.g., OTP via SMS/Email).
4. **Onboard:** Initial profile setup, selecting interests, and joining neighborhoods.
5. **Dashboard:** The main interactive hub, providing access to all features.

Key UI Components:

- **InfoHub:** Displays real-time weather forecasts and curated news (medical, geopolitical, climatic) fetched from the Python API.
- **Incident Reporting:** A form for users to submit detailed incident reports, which are then sent to the Python API.
- **Donation Gateway:** An interface for M-Pesa donations, interacting with the Python API for STK Push initiation.
- **Community Feed:** Displays posts, comments, and alerts, allowing for social interaction.
- **Patrol Schedule:** Shows upcoming and ongoing patrols, managed via the Java API.

5.4 Backend Implementation Details

5.4.1 Python Flask API

The Python API is structured following a production-grade pattern with dedicated layers for controllers, services, and repositories.

- **Controllers:** Handle incoming HTTP requests, validate input, and delegate to services. For instance, the `IncidentController` will receive incident reports.

```
# python-api/app/controllers/incident_controller.py
from flask import Blueprint, request, jsonify
from app.services.incident_service import IncidentService

incident_bp = Blueprint('incident', __name__)
incident_service = IncidentService()

@incident_bp.route('/incidents', methods=['POST'])
def report_incident():
    data = request.get_json()
    # Basic validation
    if not data or not all(key in data for key in ['incident_type',
    'description', 'location']):
        return jsonify({'message': 'Missing required fields'}), 400

    # Enqueue work for asynchronous processing
    incident_service.enqueue_incident_report(data)
    return jsonify({'message': 'Incident report received and being
    processed'}), 202
```

- **Services:** Contain the business logic, interacting with repositories and external services. The `IncidentService` will enqueue incident reports for asynchronous processing.
- **Repositories:** Abstract database interactions, providing methods for CRUD operations on models.
- **Middlewares:** For request pre-processing (e.g., authentication, logging).
- **Validators:** For complex input validation.
- **Jobs:** For background tasks (e.g., processing enqueued incident reports).

5.4.2 Java Spring Boot API

The Java API provides robust, secure services for managing core community data. It adheres to standard Spring Boot conventions.

- **Package Structure:** `com.lindajamii.controller`, `com.lindajamii.service`, `com.lindajamii.repository`, `com.lindajamii.model`, `com.lindajamii.config`.

- **Patrol Scheduling Controller:** Manages the creation, updating, and viewing of patrol schedules.

```
// java-
api/src/main/java/com/lindajamii/controller/PatrolController.java
package com.lindajamii.controller;

import com.lindajamii.model.Patrol;
import com.lindajamii.service.PatrolService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.UUID;

@RestController
@RequestMapping("/api/patrols")
public class PatrolController {

    @Autowired
    private PatrolService patrolService;

    @GetMapping
    @PreAuthorize("hasAnyRole(\"'USER'\", \"'ADMIN'\")")
    public ResponseEntity<List<Patrol>> getAllPatrols() {
        return ResponseEntity.ok(patrolService.findAll());
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole(\"'USER'\", \"'ADMIN'\")")
    public ResponseEntity<Patrol> getPatrolById(@PathVariable UUID id)
    {
        return patrolService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    @PreAuthorize("hasRole(\"'ADMIN'\")")
    public ResponseEntity<Patrol> createPatrol(@RequestBody Patrol
patrol) {
        return ResponseEntity.ok(patrolService.save(patrol));
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole(\"'ADMIN'\")")
```

```

    public ResponseEntity<Patrol> updatePatrol(@PathVariable UUID id,
    @RequestBody Patrol patrol) {
        return ResponseEntity.ok(patrolService.update(id, patrol));
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole(\"ADMIN\")")
    public ResponseEntity<Void> deletePatrol(@PathVariable UUID id) {
        patrolService.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}

```

- **Spring Security:** Integrated for authentication (e.g., JWT) and authorization (`@PreAuthorize` annotations) to secure endpoints based on user roles.

5.4.3 C Micro-Service

The C micro-service provides highly efficient, low-level functionalities. It exposes simple HTTP endpoints.

- **Endpoints:**
 - `/health` : Returns the service status.
 - `/hash` : Computes a DJB2 hash for a given string, demonstrating high-performance utility.
 - `/stats` : Provides basic service statistics (e.g., uptime, request count).
- **Python Integration Example:** The Python API can interact with the C service using standard HTTP client libraries.

```
# python-api/app/services/c_service_client.py
import requests

C_SERVICE_BASE_URL = "http://localhost:8090"

def get_c_service_health():
    try:
        response = requests.get(f"{C_SERVICE_BASE_URL}/health")
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        return {"error": str(e), "status": "unreachable"}

def get_djb2_hash(text):
    try:
        response = requests.post(f"{C_SERVICE_BASE_URL}/hash", json=
{'text': text})
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        return {"error": str(e), "hash": None}
```

5.5 External API Integrations

- **M-Pesa Daraja API (STK Push):** Integrated into the Python API to facilitate secure mobile money donations. This involves initiating an STK Push to the user's phone and handling the callback for transaction status.
- **Weather API (OpenWeatherMap):** Fetches localized weather forecasts, providing real-time environmental data to the InfoHub.
- **News API (NewsAPI):** Curates news articles related to medical, geopolitical, and climatic changes, offering relevant global and local updates.

CHAPTER SIX: QUEUES AND ASYNCHRONOUS PROCESSING

6.1 Decoupling Services

In a microservices architecture, it is crucial to decouple services to improve fault tolerance, scalability, and responsiveness. LindaJamii utilizes message queues to achieve this decoupling, particularly for tasks that are time-consuming or can be processed independently without blocking the main request flow.

Benefits of Decoupling with Queues:

- **Improved Responsiveness:** Frontend requests (e.g., incident reporting) can return immediately (202 Accepted) while the actual processing happens in the background.
- **Increased Reliability:** If a downstream service is temporarily unavailable, messages remain in the queue and can be processed once the service recovers, preventing data loss.
- **Scalability:** Work can be distributed across multiple workers consuming from the queue, allowing for horizontal scaling of processing capacity.

6.2 Message Visibility, Acknowledgements, and Retries

Message queues provide mechanisms to ensure reliable message delivery and processing:

- **Message Visibility:** When a message is consumed by a worker, it becomes invisible to other consumers for a configurable period (visibility timeout). This prevents multiple workers from processing the same message.
- **Acknowledgements (Acks):** After a message is successfully processed, the consumer sends an acknowledgment to the queue. This signals that the message can be safely deleted from the queue.

- **Retries and Dead-Letter Queues (DLQs):** If a message fails to process (e.g., due to an error in the consumer or a timeout), it can be returned to the queue for retry. After a configured number of retries, if processing still fails, the message is moved to a Dead-Letter Queue (DLQ) for manual inspection and debugging, preventing it from blocking the main queue.

6.3 Incident Alert Processing Pipeline

LindaJamii will adapt a video-upload-style pipeline for processing incident alerts, ensuring that even complex incidents are handled efficiently and reliably.

Pipeline Steps:

1. **User Reports Incident:** A user submits an incident report via the frontend.
2. **Python API: Enqueue Incident:** The Python API receives the report, performs initial validation, and immediately returns a `202 Accepted` response. The full incident data is then pushed to an `Incident Queue`.
3. **Incident Processor (Worker):** A dedicated worker service (e.g., a Python background job) consumes messages from the `Incident Queue`. This worker performs:
 - **Geo-coding:** Resolving location details.
 - **AI Analysis (Future Extension):** Categorizing incident severity or identifying patterns.
 - **Database Storage:** Persisting the incident details in the database.
 - **Notification Trigger:** If the incident requires immediate attention, it enqueues a message to an `Alert Queue`.
4. **Alert Dispatcher (Worker):** Another worker consumes from the `Alert Queue` and dispatches notifications to affected users via various channels (SMS, email, push notifications).

6.4 Queue Technologies Comparison

Choosing the right queue technology depends on specific requirements for throughput, durability, and complexity.

Feature / Technology	Amazon SQS	RabbitMQ	Apache Kafka
Type	Managed Service	Message Broker	Distributed Streaming Platform
Durability	High	High	Very High (replicated logs)
Ordering	Best-effort (Standard), FIFO (with FIFO queues)	Guaranteed (per queue)	Guaranteed (per partition)
Scalability	Highly scalable (auto-scaling)	Scalable (clustering)	Extremely scalable (distributed)
Complexity	Low	Medium	High
Message Size	Up to 256 KB	Unlimited (streams)	Up to 1 MB (configurable)
Use Case	Decoupling microservices, simple task queues	Complex routing, enterprise messaging	Real-time data pipelines, event sourcing

For LindaJamii, given the need for reliable incident processing and potential for high-volume alerts, a robust queueing system is essential. Initially, a simpler solution like **RabbitMQ** might be suitable for its ease of deployment and guaranteed message delivery for task queues. For future high-throughput event streaming (e.g., real-time location updates), **Apache Kafka** would be a strong consideration.

CHAPTER SEVEN: DEPLOYMENT AND ROBUSTNESS

7.1 Environment Configuration

Effective environment configuration is crucial for managing different deployment stages (development, staging, production) and securing sensitive information. LindaJamii will utilize environment variables for all sensitive data and configuration parameters.

- **Sensitive Data:** API keys (M-Pesa, Weather, News), database credentials, JWT secrets.
- **Configuration:** Database connection strings, service ports, logging levels.

Example `.env` file structure:

```
# Python API
FLASK_APP=run.py
FLASK_ENV=development
PYTHON_API_PORT=5050
DATABASE_URL=postgresql://user:password@host:port/database
MPESA_CONSUMER_KEY=your_mpesa_consumer_key
MPESA_CONSUMER_SECRET=your_mpesa_consumer_secret
MPESA_SHORTCODE=your_mpesa_shortcode
MPESA_PASSKEY=your_mpesa_passkey
OPENWEATHER_API_KEY=your_openweathermap_api_key
NEWSAPI_API_KEY=your_newsapi_api_key
JWT_SECRET_KEY=super_secret_jwt_key

# Java API
SPRING_DATASOURCE_URL=jdbc:postgresql://host:port/database
SPRING_DATASOURCE_USERNAME=user
SPRING_DATASOURCE_PASSWORD=password
JAVA_API_PORT=8080

# C Service
C_SERVICE_PORT=8090
```

7.2 Local Startup Commands

For local development, each service can be started independently:

```
# 1. Start C Micro-Service
cd lindajamii/c-service
make
./lindajamii-service &

# 2. Start Java Spring Boot API (requires Java 17 and Maven)
cd lindajamii/java-api
mvn clean package -DskipTests
java -jar target/lindajamii-api-1.0.0.jar &

# 3. Start Python Flask API (requires Python 3.11 and pip)
cd lindajamii/python-api
pip install -r requirements.txt
FLASK_APP=run.py FLASK_ENV=development FLASK_RUN_PORT=5050 flask run &

# 4. Start Frontend (requires Node.js for development, or simple HTTP server
for production build)
cd lindajamii/web
# For development (if using a build tool like Vite/Webpack):
# npm install && npm start
# For serving static files:
python3 -m http.server 3000

# Access frontend at http://localhost:3000
```

7.3 Cascading Health Checks

To ensure the overall health of the LindaJamii platform, a system of cascading health checks will be implemented. This involves each service reporting its own health and also checking the health of its dependencies.

- **C Service Health:** Simple `/health` endpoint.
- **Python API Health:** `/api/health` endpoint that checks its database connection, external API connectivity (M-Pesa, Weather, News), and the C service health.
- **Java API Health:** `/api/health` endpoint that checks its database connection and any internal services.
- **Frontend Health:** Periodically pings backend `/health` endpoints to display system status on the dashboard.

This cascading approach provides a clear picture of system health and helps in quickly identifying points of failure.

7.4 JWT + bcrypt Security

Security is paramount for LindaJamii. The platform will implement industry-standard security measures:

- **JSON Web Tokens (JWT):** For stateless authentication. Upon successful login, the Java API will issue a JWT to the client. This token will be included in subsequent requests to authenticate the user. JWTs are signed to prevent tampering.
- **bcrypt:** For secure password hashing. User passwords will never be stored in plain text. Instead, they will be hashed using bcrypt, a strong, adaptive hashing algorithm, making them resistant to brute-force attacks.
- **Spring Security:** The Java API will leverage Spring Security for comprehensive authentication and authorization, including role-based access control (`@PreAuthorize`).

7.5 Error Envelope Convention

To provide consistent and informative error responses across all APIs, a standardized error envelope convention will be adopted.

Example Error Response:

```
{
  "status": "error",
  "code": 400,
  "message": "Invalid input data",
  "details": [
    {"field": "incident_type", "error": "Incident type is required"},
    {"field": "location", "error": "Location cannot be empty"}
  ],
  "timestamp": "2026-05-14T10:30:00Z"
}
```

This convention ensures that clients can reliably parse and display error messages, improving the user experience and simplifying debugging.

7.6 Scaling Guidelines

LindaJamii is designed with scalability in mind:

- **Horizontal Scaling:** All stateless services (Python API, Java API, C Service) can be scaled horizontally by running multiple instances behind a load balancer.
- **Database Scaling:** The database can be scaled vertically (more powerful server) or horizontally (read replicas, sharding for very large datasets).
- **Queueing System:** Utilizing a robust queueing system (like RabbitMQ or Kafka) allows for asynchronous processing and buffering of requests, preventing backpressure on services.
- **Caching:** Implementing caching layers (e.g., Redis) for frequently accessed but rarely changing data can significantly reduce database load.

7.7 CI/CD Pipeline

A Continuous Integration/Continuous Deployment (CI/CD) pipeline will automate the build, test, and deployment processes, ensuring rapid and reliable delivery of new features and bug fixes.

Typical CI/CD Workflow:

1. **Code Commit:** Developers push code to the GitHub repository.
2. **Continuous Integration (CI):**
 - Automated tests (unit, integration) are run.
 - Code quality checks (linters, static analysis).
 - Build artifacts (JARs, Docker images) are created.
3. **Continuous Deployment (CD):**
 - Artifacts are deployed to a staging environment for further testing.
 - After successful staging tests, artifacts are deployed to production.

7.8 Pre-Launch Security Checklist

Before launching LindaJamii to a wider audience, a comprehensive security checklist will be followed:

- ☐ All environment variables are securely configured and not hardcoded.
- ☐ All API endpoints are protected by authentication and authorization.
- ☐ Input validation is implemented on all user-facing forms and API endpoints.
- ☐ Passwords are hashed using bcrypt.
- ☐ HTTPS is enforced for all communication.
- ☐ Dependencies are up-to-date to mitigate known vulnerabilities.
- ☐ Logging and monitoring are in place to detect suspicious activity.
- ☐ Regular security audits and penetration testing (if resources allow).

CHAPTER EIGHT: CONCLUSION AND FUTURE WORK

8.1 Conclusion

LindaJamii represents a significant step towards fostering safer and more informed communities through technology. By integrating a multi-language microservices architecture with a modern web frontend, the platform successfully addresses the challenges of fragmented communication, delayed incident response, and opaque community funding. The implementation of real-time incident reporting, an InfoHub for critical environmental and geopolitical updates, and a secure M-Pesa donation gateway provides a comprehensive solution that is both technically robust and socially impactful. The project demonstrates the power of a polyglot approach in building scalable and maintainable distributed systems, offering a blueprint for future community-centric digital initiatives.

8.2 Future Work

While LindaJamii provides a strong foundation, several areas are identified for future enhancements:

- **Mobile Native Applications:** Developing dedicated iOS and Android applications to provide a more optimized mobile experience and leverage device-specific features (e.g., GPS for incident location).
- **Advanced Analytics and AI:** Integrating machine learning models for predictive incident analysis, sentiment analysis on community posts, or automated anomaly detection.
- **Real-time Communication:** Implementing WebSockets for instant, bi-directional communication (e.g., live chat for incidents, real-time updates on patrol movements).

- **Gamification:** Introducing elements of gamification to encourage community participation and engagement.
- **IoT Integration:** Connecting with IoT devices (e.g., smart cameras, environmental sensors) to automatically report incidents or environmental changes.
- **Multi-language Support:** Extending the frontend and backend to support multiple human languages for broader accessibility.

REFERENCES

1. Sommerville, I. (2015). *Software Engineering*. 10th ed. Pearson.
2. Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
3. Safaricom Developer Portal. (2023). *Daraja API Documentation*. [Online] Available at: <https://developer.safaricom.co.ke/> [Accessed 14 May 2026].
4. OpenWeatherMap. (2023). *Weather API Documentation*. [Online] Available at: <https://openweathermap.org/api> [Accessed 14 May 2026].
5. NewsAPI. (2023). *News API Documentation*. [Online] Available at: <https://newsapi.org/docs> [Accessed 14 May 2026].
6. Spring Framework Documentation. (2023). *Spring Boot Reference Guide*. [Online] Available at: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> [Accessed 14 May 2026].
7. Pallets Projects. (2023). *Flask Documentation*. [Online] Available at: <https://flask.palletsprojects.com/> [Accessed 14 May 2026].
8. Apache Kafka. (2023). *Apache Kafka Documentation*. [Online] Available at: <https://kafka.apache.org/documentation> [Accessed 14 May 2026].
9. RabbitMQ. (2023). *RabbitMQ Documentation*. [Online] Available at: <https://www.rabbitmq.com/documentation.html> [Accessed 14 May 2026].
10. Amazon Web Services. (2023). *Amazon SQS Documentation*. [Online] Available at: <https://aws.amazon.com/sqs/> [Accessed 14 May 2026].

APPENDIX

Appendix A: Database Schema (SQL DDL)

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  username VARCHAR(255) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  phone_number VARCHAR(20),  
  profile_picture_url TEXT,  
  bio TEXT,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  is_verified BOOLEAN DEFAULT FALSE,  
  role VARCHAR(50) DEFAULT 'user'  
);  
  
CREATE TABLE posts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  content TEXT NOT NULL,  
  media_url TEXT,  
  post_type VARCHAR(50) DEFAULT 'text',  
  location VARCHAR(255),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE comments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  content TEXT NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE followers (  
  follower_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
```

```
followed_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
PRIMARY KEY (follower_id, followed_id)  
);
```

```
CREATE TABLE likes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE (post_id, user_id) -- A user can only like a post once  
);
```

```
CREATE TABLE chats (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  chat_name VARCHAR(255),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE messages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  chat_id UUID NOT NULL REFERENCES chats(id) ON DELETE CASCADE,  
  sender_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  content TEXT NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE notifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  type VARCHAR(50) NOT NULL,  
  content TEXT NOT NULL,  
  is_read BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE incidents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  reporter_id UUID NOT NULL REFERENCES users(id) ON DELETE SET NULL,  
  incident_type VARCHAR(100) NOT NULL,  
  description TEXT,  
  location VARCHAR(255),  
  status VARCHAR(50) DEFAULT 'reported',  
  severity VARCHAR(50) DEFAULT 'medium',  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```

        updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE alerts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    alert_type VARCHAR(100) NOT NULL,
    message TEXT NOT NULL,
    geographical_scope VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE neighbourhoods (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) UNIQUE NOT NULL,
    description TEXT,
    location_coords VARCHAR(255), -- e.g., "-1.286389, 36.817223"
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE patrols (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    neighbourhood_id UUID NOT NULL REFERENCES neighbourhoods(id) ON DELETE
    CASCADE,
    patrol_leader_id UUID REFERENCES users(id) ON DELETE SET NULL,
    start_time TIMESTAMP WITH TIME ZONE NOT NULL,
    end_time TIMESTAMP WITH TIME ZONE NOT NULL,
    status VARCHAR(50) DEFAULT 'scheduled',
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

Appendix B: Frontend 5-Step App Flow Diagrams

(Diagrams for Splash, Sign Up, Verify, Onboard, and Dashboard flows would be included here, possibly using Mermaid or similar tools for visual representation.)

Appendix C: API Endpoints Summary

Service	Endpoint	Method	Description	Authentication
Python API	/api/incidents	POST	Report a new incident	JWT Required
	/api/incidents	GET	Get all incidents	JWT Required
	/api/mpesa/stkpush	POST	Initiate M-Pesa STK Push	JWT Required
	/api/mpesa/callback	POST	M-Pesa callback URL	None
	/api/info/weather	GET	Get local weather forecast	JWT Optional
	/api/info/news	GET	Get curated news	JWT Optional
Java API	/api/auth/login	POST	User login	None
	/api/auth/register	POST	User registration	None
	/api/patrols	GET	Get all patrols	JWT Required (USER/ADMIN)
	/api/patrols	POST	Create new patrol	JWT Required (ADMIN)
	/api/neighbourhoods	GET	Get all neighbourhoods	JWT Optional
C Service	/health	GET	Service health check	None
	/hash	POST	Compute DJB2 hash	None

GLOSSARY

- **API (Application Programming Interface):** A set of definitions and protocols for building and integrating application software.
- **Asynchronous Processing:** A method of processing where a task can run in the background without blocking the main application thread.
- **bcrypt:** A password-hashing function designed to be slow, making brute-force attacks computationally expensive.
- **CI/CD (Continuous Integration/Continuous Deployment):** A set of practices that enable rapid and reliable delivery of software by automating the build, test, and deployment processes.
- **Controller:** In an MVC (Model-View-Controller) or similar architectural pattern, a component that processes user input and orchestrates the response.
- **CRUD (Create, Read, Update, Delete):** The four basic functions of persistent storage.
- **Daraja API:** Safaricom's M-Pesa API, allowing integration of M-Pesa services into other applications.
- **Database Schema:** The formal description of how data is organized in a database.
- **Decoupling:** Reducing the interdependencies between components of a system.
- **Distributed System:** A system whose components are located on different networked computers, which communicate and coordinate their actions by passing messages to one another.
- **DLQ (Dead-Letter Queue):** A queue where messages are sent after failing to be processed successfully a certain number of times.
- **ERD (Entity-Relationship Diagram):** A diagram that illustrates the relationships between entities in a database.
- **Flask:** A micro web framework for Python.
- **Frontend:** The part of a website or application that users interact with directly.

- **JWT (JSON Web Token):** A compact, URL-safe means of representing claims to be transferred between two parties.
- **Microservices Architecture:** An architectural style that structures an application as a collection of loosely coupled services.
- **M-Pesa:** A mobile phone-based money transfer, financing, and microfinancing service, launched by Vodafone and Safaricom.
- **Polyglot Architecture:** The practice of building applications using a mix of programming languages and technologies.
- **PostgreSQL:** A powerful, open-source object-relational database system.
- **Queue:** A data structure that stores messages temporarily before they are processed.
- **Repository:** A layer in an application that abstracts data access logic, providing a clean API for interacting with data sources.
- **REST (Representational State Transfer):** An architectural style for designing networked applications.
- **Service:** A component that encapsulates business logic and interacts with repositories and other services.
- **SPA (Single Page Application):** A web application that loads a single HTML page and dynamically updates that page as the user interacts with the app.
- **Spring Boot:** An open-source, Java-based framework used to create stand-alone, production-grade Spring applications.
- **STK Push:** A feature of M-Pesa Daraja API that sends a prompt to the customer's phone to enter their M-Pesa PIN to complete a transaction.
- **UML (Unified Modeling Language):** A standardized general-purpose modeling language in the field of object-oriented software engineering.
- **Visibility Timeout:** A period during which a message is hidden from other consumers after being retrieved from a queue.